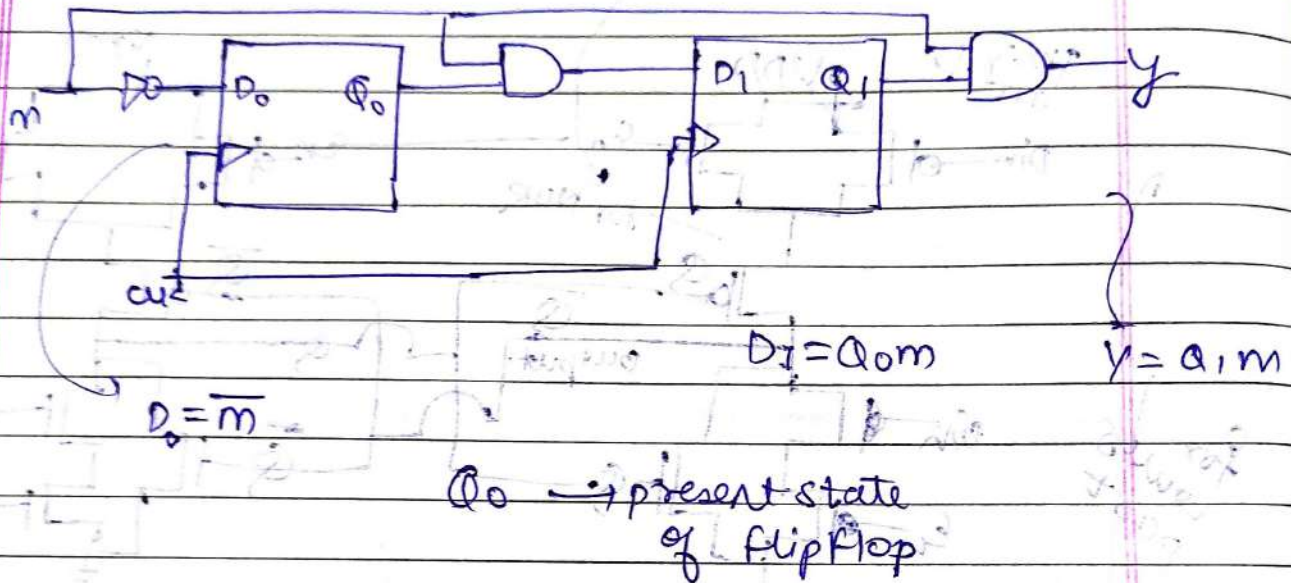


FSM Interview Q. A: _____

Prerna Agarwal

Q What is Present state & what type of logic is applied in Present state?

→ Present state means output of FlipFlops



✓ sequential logic is needed

```

always_ff (begin @ (posedge clk) begin
    if (rst)
        currentstate <= Idle;
    else
        currentstate <= nextstate;
end
    
```

Q Next state logic ??

→ It is combinational logic because in order to find Q_1 → Q_0 has to travel through combinational path

always_comb begin
case (currentstate)

A: nextstate = x ? B : C

endcase

end

or

assign nextstate

= (A & (x==1))

Q Output logic?

→ It is also combinational logic

assign out = (currentstate == D)

or

case (currentstate)

S0: out = 0;

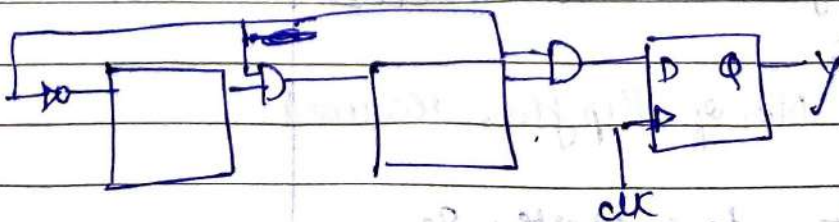
S1: out = 1;

endcase

Q If you want your output synchronous with clock what will you do??

→

add 1 D flipFlop before output signal



What is FSM?

- It is Sequential model that represents a system with finite no. of states and system behaviour depends on present state & input and state transition occur synchronously with a clock.

Mealy

Mooe

- | | |
|--|--|
| <p>→ output $\rightarrow$ present state + present input</p> <p>→ responds faster becoz of input changes</p> <p>→ output may have glitches becoz output changes immediately with input change even b/w clock edges</p> | <p>output = present state</p> <p>slower becoz wait for</p> |
|--|--|

Which machine consumes more power & area

Mealy

Mooe

- | | |
|--|--|
| <p>uses few states (less hardware) becoz</p> <p><u>output</u></p> <p>→ less No. of flip flops required</p> <p>→ Due to few states & reduced combination logic $\therefore$ low power consumption</p> | |
|--|--|

Q How do you optimize FSM?

→ FSM optimization refers to reducing hardware complexity

- ✓ Eliminate redundant states
- ✓ Remove unreachable states
- ✓ choose efficient state encoding

① states that produce identical output & have identical transitions for all input

Identify this by state diagram

Adv: Reduced no. of Flip Flops ↓ ↓
combinational logic ↓ ↓

eg:-

Q Overlapping & Non-Overlapping Sequence Detectors & why Both needed?

→

Overlapping sequence detectors are faster Bcoz once detected then bits from previous sequence can be reused to detect next sequence pattern.

Non overlapping are simpler once sequence detected then detector resets & doesn't reuse bits from previous sequence

Q why Moore ~~has~~ has $n+1$ states??

→

Moore output depends only on state so additional state is needed to assert detection output.

eg: 101 detector

suppose input passed as 1 0 1 After receiving Last 1
State $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$

↓ ↓ ↓ then FSM enters S_3
Output 0 0 0 1

output depends on current state not on input

Q Difference b/w FSM & Counters

Counter $\rightarrow$ follows a fixed counting sequence (0, 1, 2, ...)

FSM $\rightarrow$ It can move between states based on current state & inputs

Counters	FSM
$\rightarrow$ purpose is counting the numbers.	To control the system behaviour
$\rightarrow$ External inputs not required.	External inputs are required
$\rightarrow$ Flexibility is Low	Flexibility is High

Note

Every Counter is an FSM
 Becoz counter has
 ✓ states ✓ FlipFlops
 ✓ state transition

Every FSM is not a Counter.

Q Can FSM be infinite?

→ No

FSM contains limited No. of states which are determined by available flipflops

if n flipflops,

$$\text{Maximum states} = 2^n$$

Since digital hardware has a finite number of flipflops.

Q how an fsm is implemented?

FSM consists of 3 Main Blocks:

- 1) Next state logic
- 2) Present state (output of flipflop)
- 3) output logic

1. Reading the specification

2. Draw state Diagram

3. obtain state table

PS		NS		Output		
(ip)	$x=0$	$x=1$	$x=0$	$x=1$	$x=0$	$x=1$

4. Choose state Encoding
 ✓ one-hot
 ✓ Binary encoding

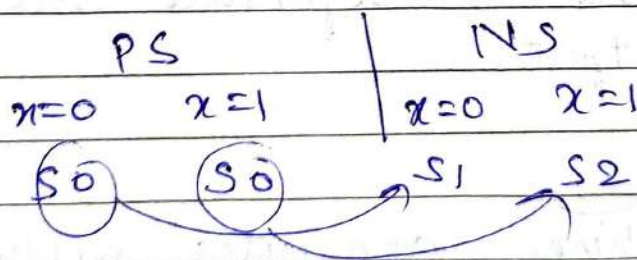
5. Through Excitation table

PS	x (FSM input)	NS	FlipFlop input	y
find flipflop input equation using Kmap				

6. Implement using flip flops, Logic gates.

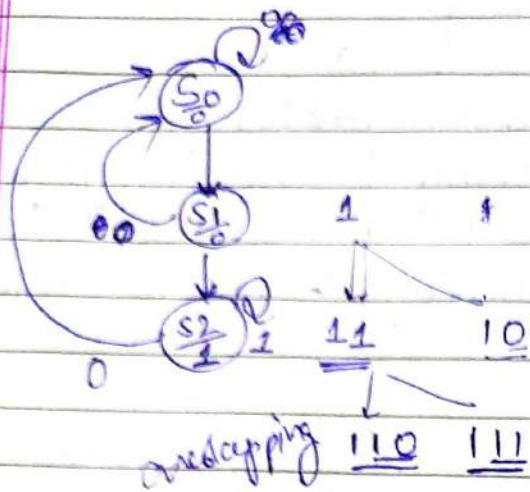
Q If Moore Machine has 4 states & 2 inputs then maximum no. of Transition?

→ Each state need 1 transition for every possible input



$$\text{Max. Transitions} = 4 \times 2 = 8$$

Q Moore machine outputs 1 only if 2 last 2 inputs are 11. Draw diagram



P.S		N.S		output	
x=0	x=1	x=0	x=1	x=0	x=1
S0	S0	S0	S1	0	0
S1	S1	S0	S2	0	0
S2	S2	S0	S2	0	1

Q Why Moore require more states than Mealy?

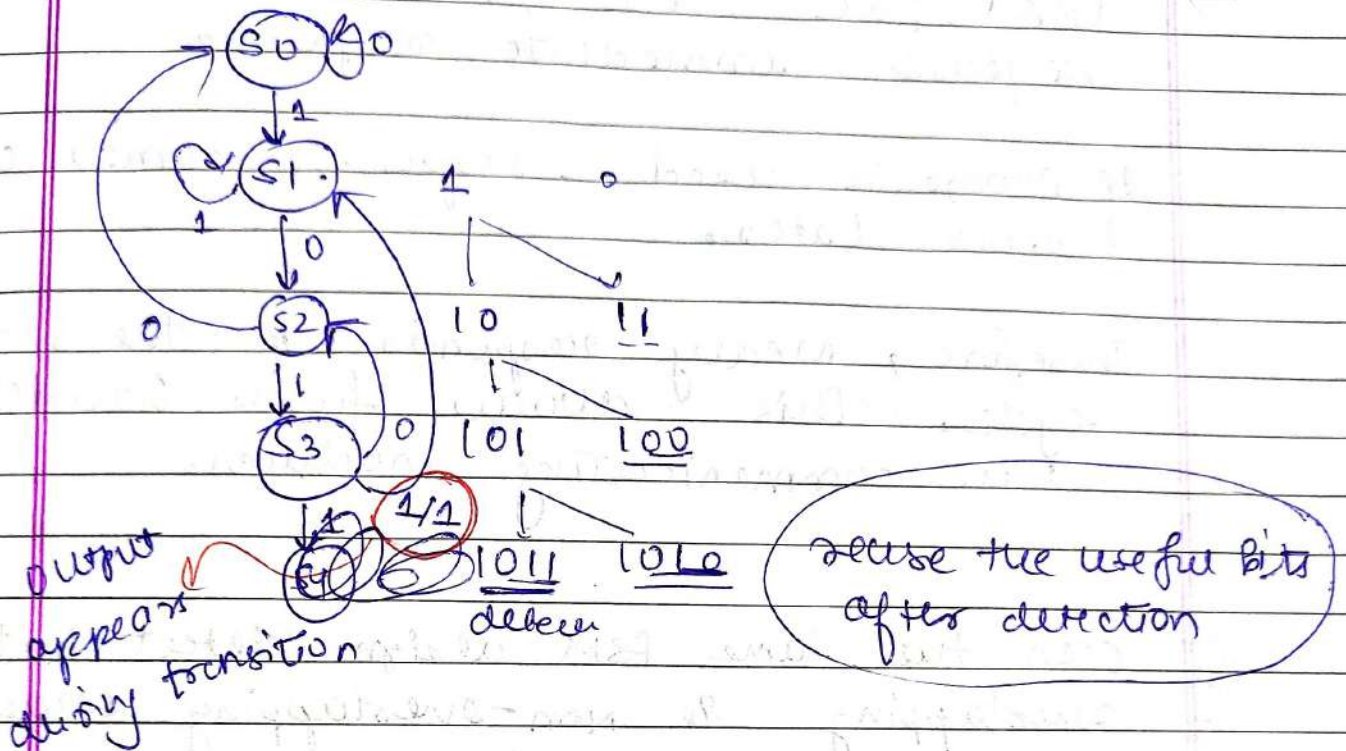
→ Moore generally requires more states because output depends on current state.

whenever a different output is needed, FSM enters a separate state to represent that output.

→ Mealy Machine generates output based on current state + input during the transition itself,

So it usually needs fewer states for same functionality.

Q 1011 detector Mealy Machine
↑ detector overlapping



Q so now output is faster ~~than~~ⁱⁿ Mealy than Moore?

Mealy

Moore

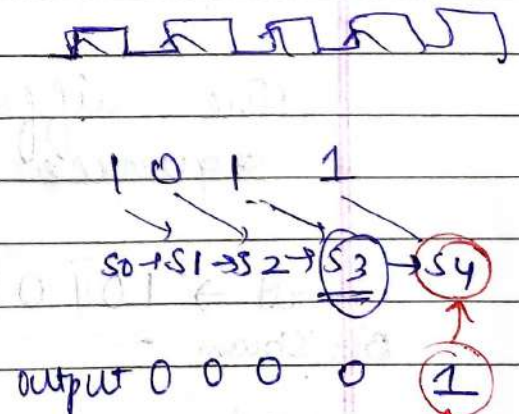
current state 101

input → 1 passed

output immediately becomes 1

✓ No extra state

✓ No extra clock



output appears 1 clock later

Q Why are Mealy machines preferred in Protocol Design?

→ UART, I2C, SPI, AXI, APB, AHB all require immediate response

If Moore is used, response comes one clock later.

Therefore, Mealy responds in the same cycle. This allows faster handshaking b/w communicating modules.

Q Can the same FSM design detect both overlapping & non-overlapping sequences? Show example?

→ YES

→ same FSM can be modified to detect both overlapping & non-overlapping sequences

The difference is what happens after a sequence is detected.

input Bit Stream	→ 10101	✓ overlapping	10101	✓ Non overlapping
output	1 1		1	

The only modification in FSM → state transition change

Q If FSM has 5 states, compare possible encoding styles.

1) Binary encoding

→ Assign binary no. to each state.
for 5 states,

$$\log_2 [5] = 3 \text{ flipflops}$$

S0 → 000

S1 → 001

S2 → 010

S3 → 011

S4 → 100

more combination logic

Beoz if to detect S3 = 011

3 bits must be decoded

min. hardware
less power
less area

2) One hot encoding

→ Each state gets 1 flipflop

for 5 state → 5 flipflops required

S0 → 00001

S1 → 00010

S2 → 00100

S3 → 01000

S4 → 10000

at a time 1 flipflop is high

↑ more flipflops

↑ more area

→ less combination logic is required
Beoz only 1 flipflop is checked hence faster.

3) Gray Encoding

Adjacent states differ by only 1 bit

$S_0 \rightarrow 000$

$S_1 \rightarrow 001$

$S_2 \rightarrow 011$

$S_3 \rightarrow 010$

$S_4 \rightarrow 110$

3 flipflops needed

less switching
less power

But state assignment is not straightforward

→ The choice of encoding depends on design goals.

Q If clock frequency = 10 MHz & FSM need 4 cycles to detect a sequence. max detection speed ??

$$\rightarrow T = \frac{1}{f} = \frac{1}{10 \times 10^6} = 100 \text{ ns}$$

→ 1 sequence takes $\rightarrow 4 \times 100 = 400 \text{ ns}$

→ detection rate $\rightarrow \frac{1}{400 \text{ ns}} = 2.5 \text{ million detections per sec}$
 $= 2.5 \text{ MHz}$

Q A sequence detector 1001 is designed using 5 states. Show how to minimize to 4 states.

→ for a Moore detector of 1001, there are no equivalent states so it can't be minimized to 4 states.

→ So, if requirement is to use only four states → then implementation must be changed to Mealy.

Q Why state minimization needed in FSM?

→ Process of removing equivalent states without changing its functionality.

Advantage

① No. of states ↓ ↓

Flip flop may / may not be reduce.

$$N=8$$

$$N=6$$

$$\log_2(8) = 3$$

$$\log_2(6) \approx 3$$

But decoding logic becomes simple.

②

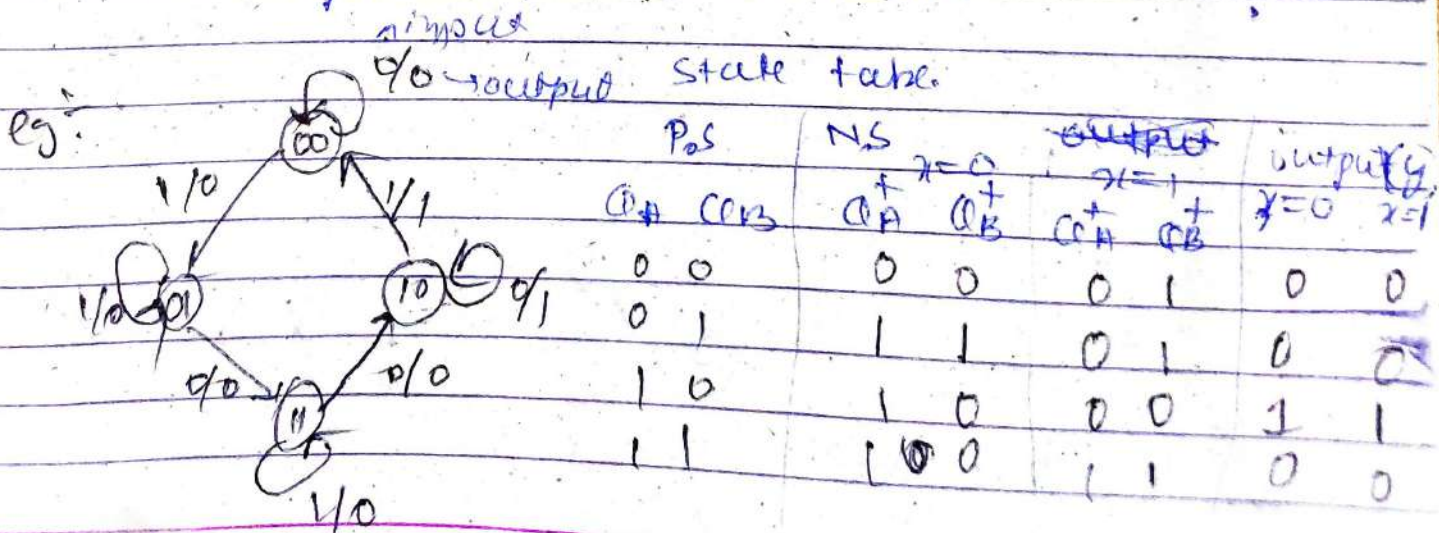
Next state logic becomes smaller
✓ less NAND / AND / OR gates

③

Propagation delay depends on FF + comb. logic
if comb. logic ↓ ↓ reduce
critical path ↓ ↓ → frequency ↑ ↑

Design Process for clocked sequential CKTs [synchronous]

- state diagram
- obtain state table
- No. of states can be reduced by state reduction method
- No state assignment
- determine No. of flip flops required
- Decide the type of flip flop used
- derive excitation table from state table.
- Obtain Output Expression & flip flops input
- Implement the circuit.



for state reduction

Next state of 2 P.S must be same

→ assigning the state

$$00 = a$$

$$01 = b$$

$$10 = c$$

$$11 \rightarrow d$$

→

$$2^{\text{FF}} = 4 (\text{states})$$

→

Q_A, Q_B used (flip-flops)

→ Excitation table.

T.F.F			P.S			N.S		T flip flop		y
clk	x	en	Q_A	Q_B	χ	Q_A^+	Q_B^+	T_A	T_B	
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	1	0	1	0
1	1	0	0	1	0	1	1	1	0	0
0	1	0	0	1	1	0	1	0	0	0
0	0	1	1	0	0	1	0	0	0	1
1	0	1	1	0	1	0	0	1	0	1
1	1	1	1	0	0	1	0	0	1	0
0	1	1	1	1	0	1	1	0	0	0

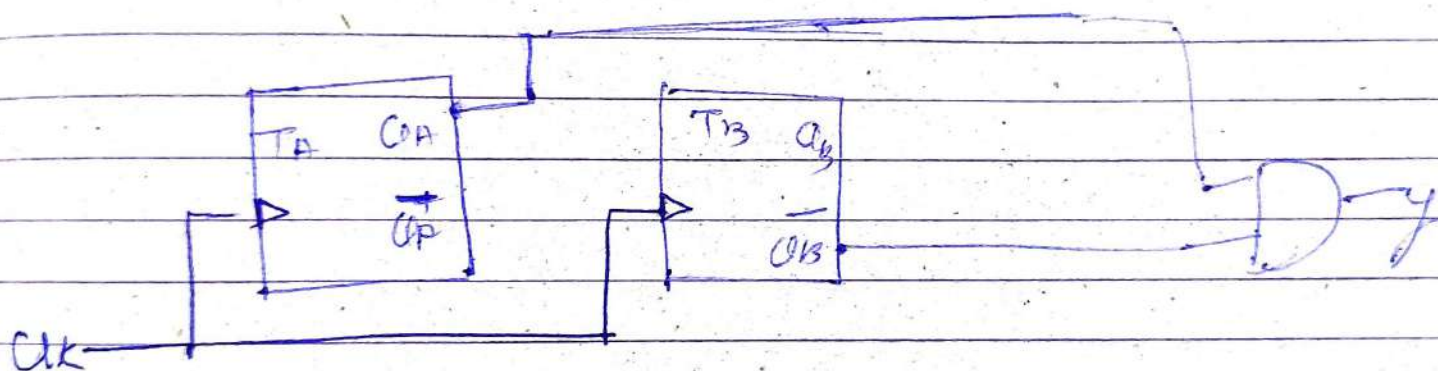
$$T_A = \overline{Q_A} \overline{Q_B} \overline{x} + Q_A \overline{Q_B} x$$

$$T_B = \overline{Q_A} \overline{Q_B} x + Q_A \overline{Q_B} \overline{x}$$

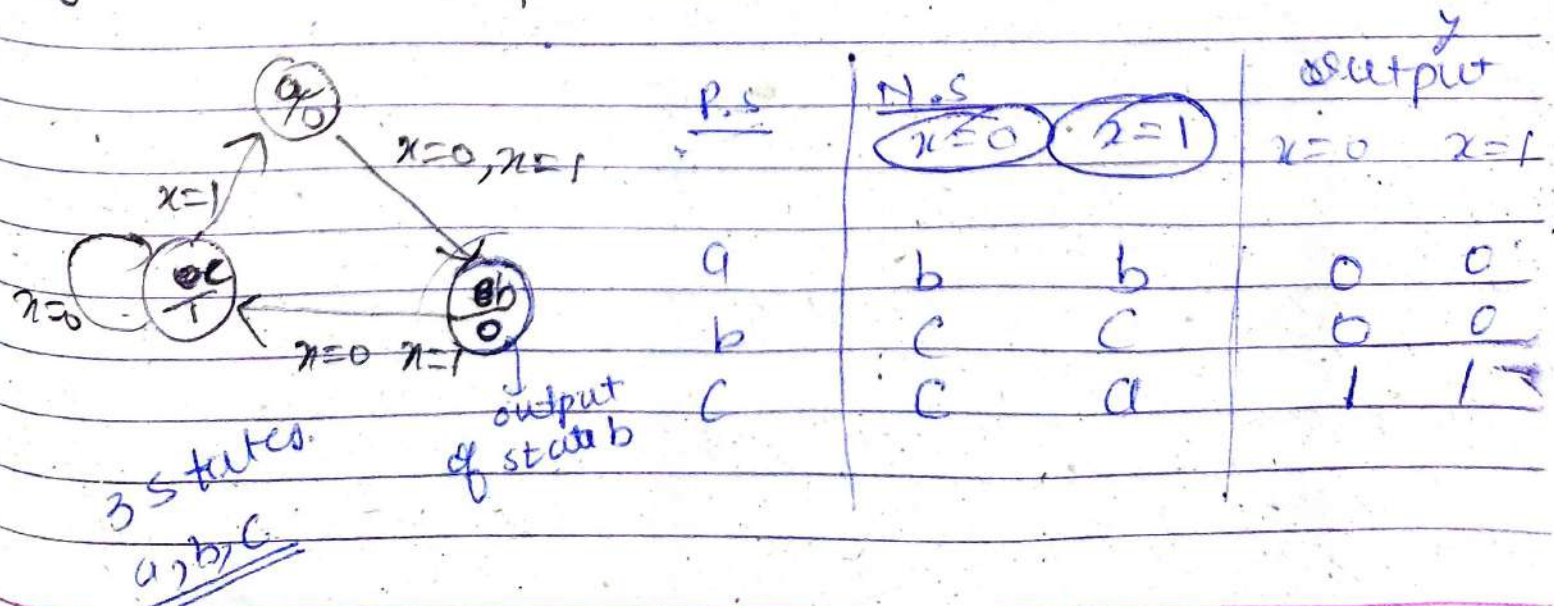
$$y = Q_A \overline{Q_B} \overline{x} + Q_A \overline{Q_B} x$$

$$= Q_A \overline{Q_B} (\overline{x} + x)$$

$$= Q_A \overline{Q_B}$$



Q (Analyze) Realize sequential ckt for state diagram



• Spee

case(i)

Sx

S1
S2

→ No possibility of state reduction
Since no 2 states, Next states & output are same.

Spee

10m

(i) fin.

(i) cor

⇒ hint

→ states are already given assigned.

→ $2^n \geq N$

$2^2 \geq 3$

∴ No. of flip flops = 2

Flip flop using → T state f.f

→ state assignment

a = 00

b = 01

c = 10

→ Excitation table

$Q \oplus Q_{next} \rightarrow T$

P.S		i/p X	N.S		FF Input		Output
QA	QB		QA	QB	TA	TB	
0	0	0	0	1	0	1	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	1	0
0	1	1	1	0	1	1	0
1	0	0	1	0	0	0	1
1	0	1	0	0	1	0	1

increasing state
treat as don't case

1 1 0 x

1 1 1

$T_A =$

$\overline{CA}$	CB	01	11	10
0	1	1	1	1
1	1	X	X	

$T_B =$

$\overline{CA}$	CB	01	11	10
0	1	1	1	1
1		X	X	

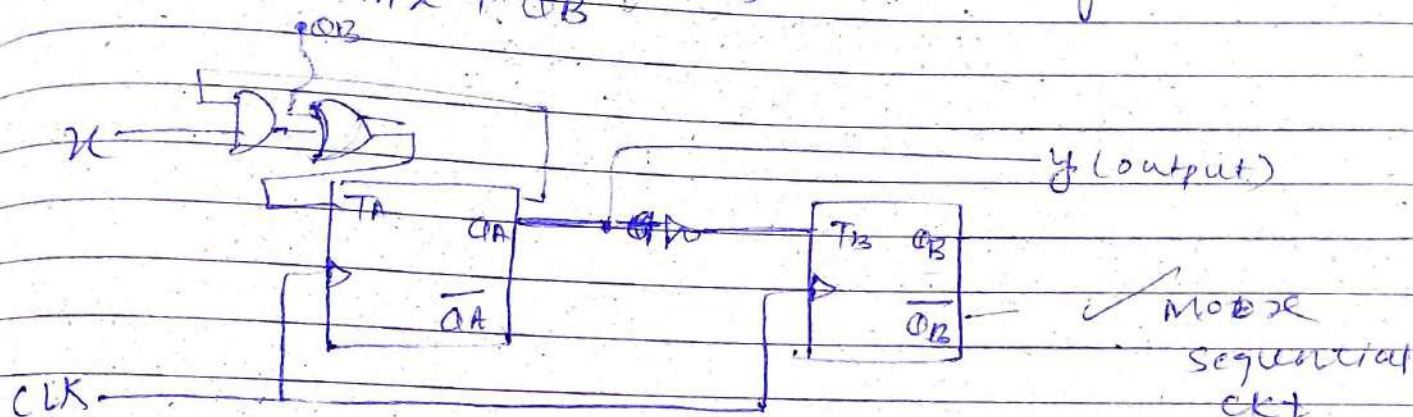
$y =$

$\overline{CA}$	CB	01	11	10
0	1	1		
1	1	1	X	X

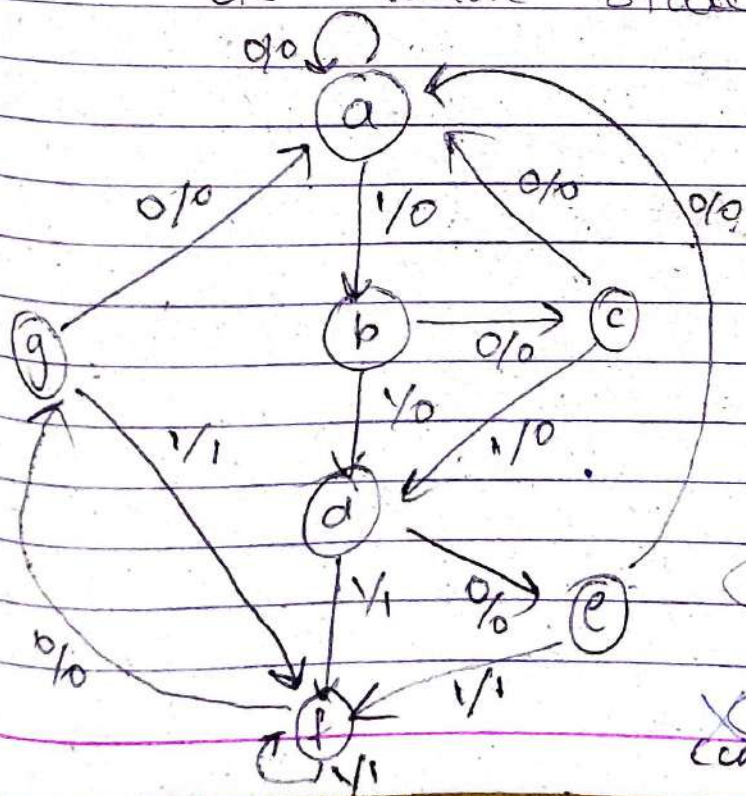
$$T_A = \overline{CA}x + CB$$

$$T_B = \overline{CA}$$

$$y = \overline{CA}$$



Q obtain reduced state tables & reduced state diagram for a sequential ckt whose state diagram



→ 7 states

$$2^n \geq 7$$

P.S	N.S		Output	
	$x=0$	$x=1$	$x=0$	$x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

cancel

e to g state can be reduced

we can cancel either e or g state

g state is cancelled. replace g with e

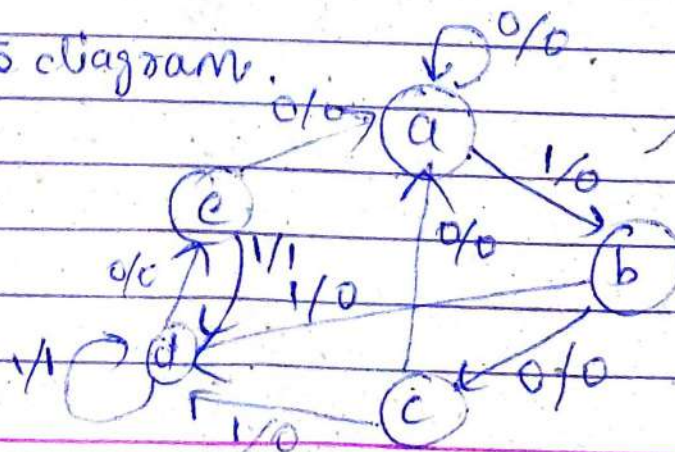
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

remove f state ; replace f → d.

	$x=0$	$x=1$	$x=0$	$x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

✓
final
reduced
state
table.

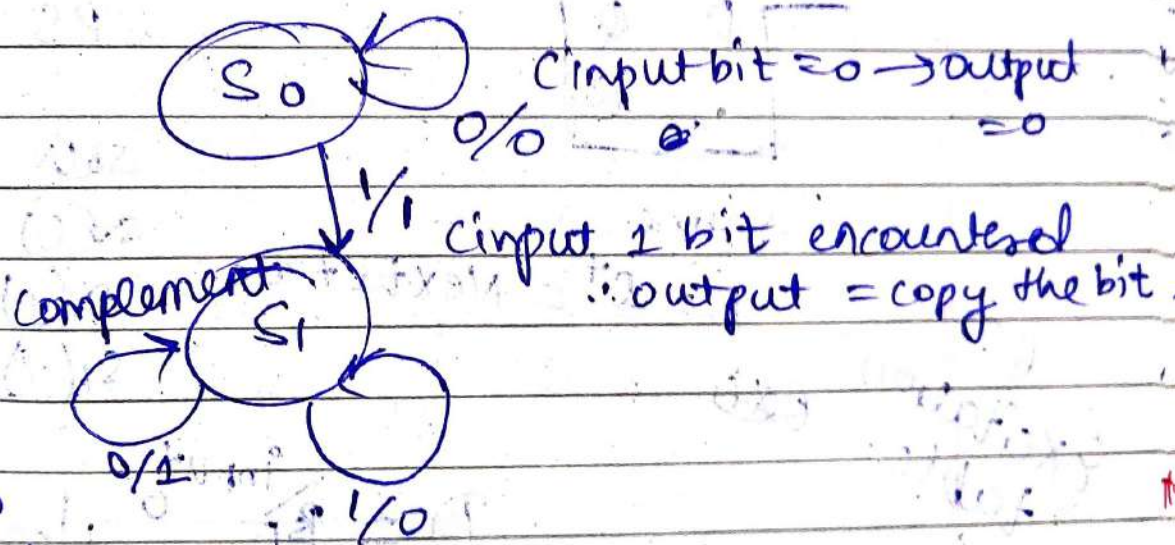
Reduced state diagram.



Q 2's CR state machine?

1101 ~~0~~
 2'sB mean 1 occurred
 0001
 reversed

110001000
 complement 00111000
 first 1 encountered
 copy



FSM design
 using flipflop

state table

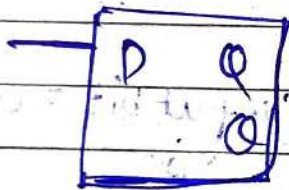
PS input N.S output

$S_0 = 0$	0	$S_0 = 0$	0
$S_0 = 0$	1	$S_1 = 1$	1
$S_1 = 1$	0	S_1	1
$S_1 = 1$	1	S_1	0

Obtain Output Eqⁿ from state table

P.S	input	Y (output)
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = 1 \text{ PS} \oplus \text{input}$$



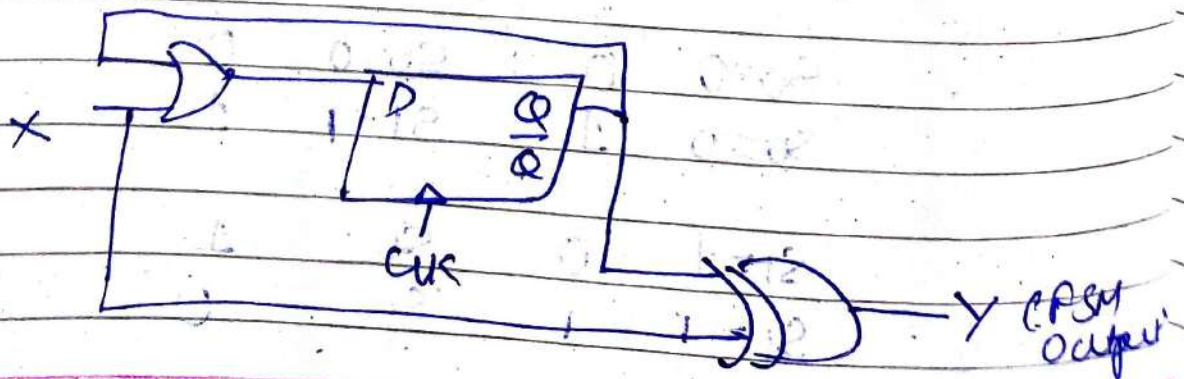
PS input (N.B) D

Since $D = Q^+ = \text{Next state}$

S(0)	0	S ₀ → 0
S(0)	1	S ₁ → 1
S(1)	0	S ₁ → 1
S(1)	1	S ₁ → 1

Excitation table

$$D = \text{input} \oplus Q = Q + X$$



Q State transition diagram for JK flipflop?

Characteristic Eqⁿ J-K

$$Q^+ = J\bar{Q}_n + \bar{K}Q_n$$

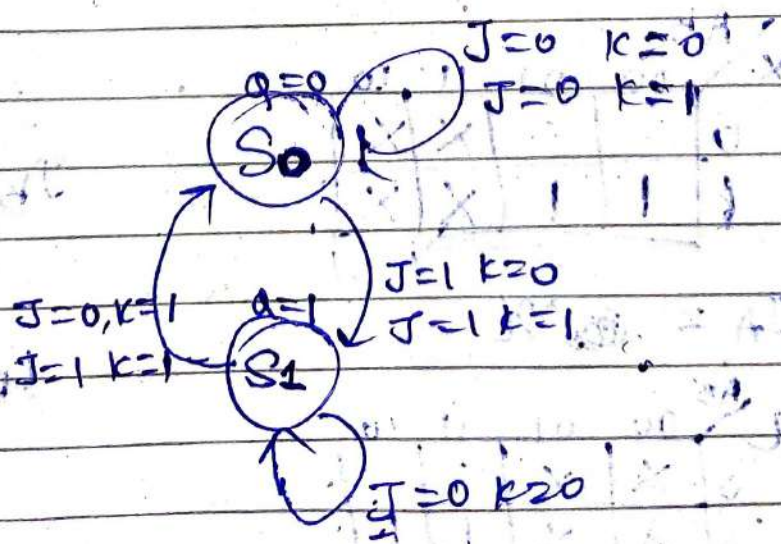
J.K flipflop $\begin{cases} \text{state 0 (Q=0)} \\ \text{state 1 (Q=1)} \end{cases}$

State transition table

PS(Q_n) J K

Q⁺ (NS) use characteristic Eqⁿ

0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



Q state Machine for detecting more than 1 ...

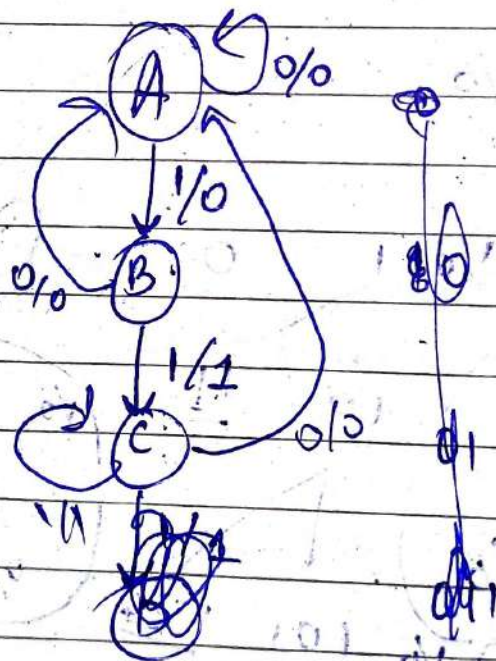
→ patterns like: 011 → (2)
101 → (2)
110 → (2)
111 → (3)

A - 000 → undetected

B - 001 → '1' one time detected
010

C - 2 (1's) detected 011, 101, 110, 111

~~D → 3 (1's) detected 111~~

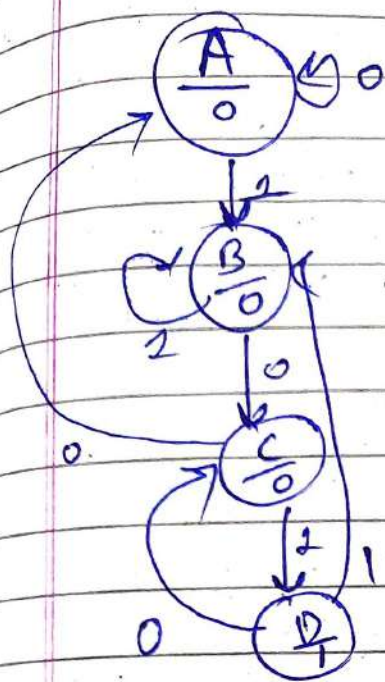


input sequence 011011101001

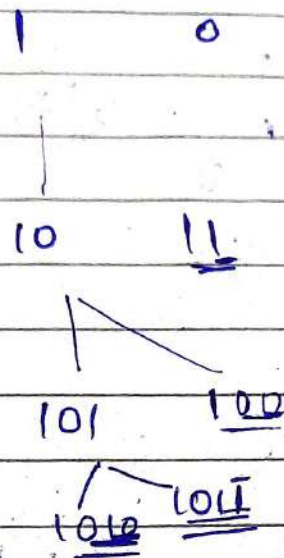
~~state~~ state seq $s_0 s_1 s_2 s_0 s_1 s_2 s_2 s_0 s_1 s_0 s_2$

output seq 001001100000

Q state machine for detecting 101



no overlapping



Q state machine, whether 1's & 0's are even or odd.

11 → even
00 →

1 → odd
0 → odd

101 → 1's → even
110 → 0's → odd

100 → odd zeros

01 → odd-odd

1100 → 1's → even
0's → even

A → (initial state)

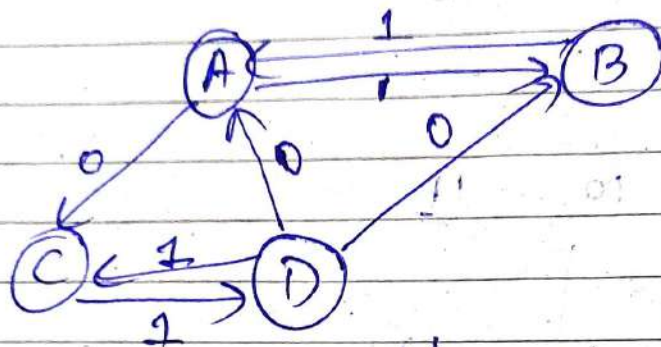
even 1's even 0's

B → odd 1's even 0's

C → even 1's odd 0's

D → odd 1's odd 0's

0101011011000
 add-add
 ds 1's



Q input sequence 101 $\rightarrow (5)_{10}$ $\xrightarrow{\text{divisible by 5}}$

out 20 remainder

out = 1

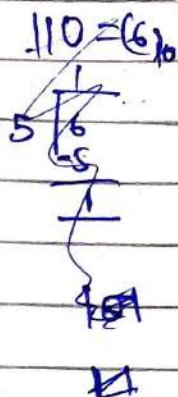
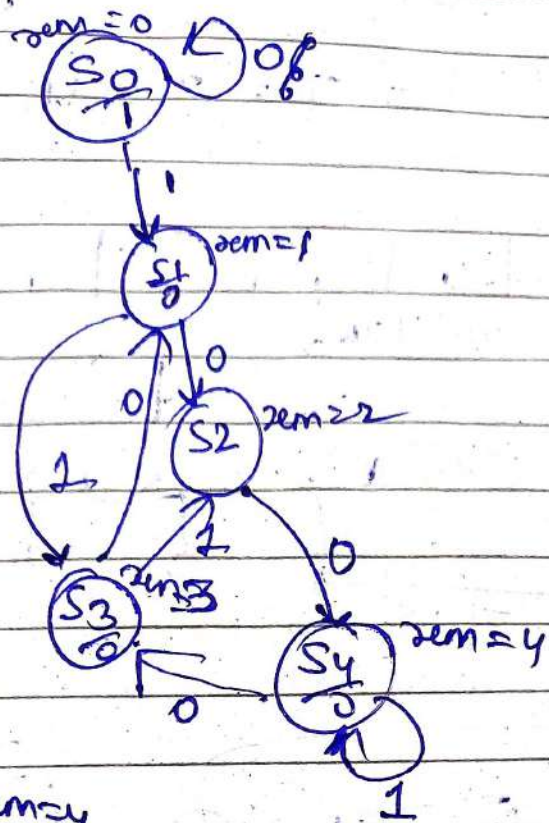
rem = 3/1
 rem = 4/100
 1011
 1010010101

OUT = 0
 OUT = 1
 OUT = 1
 OUT = 0

1
 2
 0
 0
 1

look $\rightarrow$ store remainder when divided by 5. Instead of storing entire binary numbers.

	rem	out
S0	0	1
S1	1	0
S2	2	0
S3	3	0
S4	4	0



11 rem=3
 $11 = (2)_{10}$
 $110 = (6)_{10}$

$100 \rightarrow \text{rem} = 4$

$1001 = (9)_{10} \rightarrow \text{rem} = 4$

output
 here depends
 on remainder
 only
 $\therefore$ on state.
 pe

Moore
 Model